



UNIVERSITÀ
DI TORINO

Hybrid Workflows for Artificial Intelligence

Task-based programming

Prof. Iacopo Colonnelli

✉ iacopo.colonnelli@unito.it

🌐 <https://alpha.di.unito.it/iacopo-colonnelli>

Task-based programming libraries

Within the task-based programming model, users identify and annotate functions that can be executed as **asynchronous remote tasks**. Then, the runtime system concatenates these functions' executions according to their (data) dependencies

Concurrency can be expressed either **imperatively**, by explicitly calling the library API, or **declaratively**, through annotations

Task-based libraries adopt an **implicit composition**: they infer the workflow structure automatically from the composition of the function calls

Task-based libraries build on three **primitive concepts** that serve as building blocks:

- The **asynchronous programming model** for concurrency
- The **future/promise pattern** for synchronisation
- The **data (de-)serialization** for distributed task offloading

The asynchronous programming model

The asynchronous programming model allows a program to **concurrently execute several tasks**

When a task must perform a long operation (I/O, networking, sleep, ...), it is put in a **waiting queue** and the control is transferred to a **different execution flow**

When the task is completed, an **event** is raised (similar to processor interrupts) and the task comes back in the **ready queue**

Note that asynchronous programming **differs from** parallel programming: even if asynchronous independent tasks **MAY** run in parallel



The future/promise paradigm

The asynchronous programming model is often implemented through the **future/promise** pattern: an asynchronous function immediately returns a **future** object that can be passed as argument to another asynchronous function

The concept of future appears for the first time in a 1977 paper from Henry Baker and Carl Hewitt, titled "The incremental garbage collection of processes"

“

*This paper investigates some problems associated with an argument evaluation order that we call "future" order, which is different from both call-by-name and call-by-value. In **call-by-future**, each formal parameter of a function is bound to a separate process (called a "future") dedicated to the evaluation of the corresponding argument. This mechanism allows the fully parallel evaluation of arguments to a function, and has been shown to augment the expressive power of a language*

The future/promise paradigm

- When an expression is given to the evaluator (i.e., the runtime system) by the user, a **future** for that expression is returned, which is a **promise** to deliver the value of that expression at some later time, if the expression has a value
- A **process** is created for each new future which immediately starts to work evaluating the given expression
- When the value of a future is **needed explicitly**, the evaluation process may or may not have finished:
 - If it has finished, the value is **immediately made available**
 - If not, the requesting process **is forced to wait** until it finishes
- Futures are created **recursively** in the evaluation of an expression (i.e., a program) whenever our eager evaluator encounters functional application

Example: the Python `async / await` paradigm



UNIVERSITÀ
DI TORINO

Some modern programming languages implement the **future** pattern as a primitive feature (e.g., Python, Javascript, Go)

For example, the Python language relies on the `async / await` programming model:

- An asynchronous function, called **coroutine**, is declared using the `async def` syntax
- A program can wait for the completion of an asynchronous function through the `await` keyword
- The return value of a coroutine can either be **awaited directly** or **assigned to a variable**, i.e., a future, through the `create_task` function
- Asynchronous function calls can be **concatenated**, **nested**, and **recursively implemented**

```
## Bad implementation

async def r():
    return a, b

async def s(a, b):
    return c, d, e

async def t(a):
    ...

async def u(a, b):
    ...

async def main():
    a, b = await r()
    c, d, e = await s(a, b)
    await t(c)
    await u(d, e)

if __name__ == '__main__':
    asyncio.run(main())
```

Example: the Python `async / await` paradigm



UNIVERSITÀ
DI TORINO

Some modern programming languages implement the **future** pattern as a primitive feature (e.g., Python, Javascript, Go)

For example, the Python language relies on the `async / await` programming model:

- An asynchronous function, called **coroutine**, is declared using the `async def` syntax
- A program can wait for the completion of an asynchronous function through the `await` keyword
- The return value of a coroutine can either be **awaited directly** or **assigned to a variable**, i.e., a future, through the `create_task` function
- Asynchronous function calls can be **concatenated**, **nested**, and **recursively implemented**

```
## Better implementation

async def r():
    return a, b

async def s(t):
    a, b = await t
    ...
    return c, d, e

async def t(t):
    a, _, _ = await t
    ...

async def u(t):
    _, a, b = await t
    ...

async def main():
    t1 = asyncio.create_task(r())
    t2 = asyncio.create_task(s(t1))
    t3 = asyncio.create_task(t(t2))
    t4 = asyncio.create_task(u(t3))
    await asyncio.gather(t3, t4)

if __name__ == '__main__':
    asyncio.run(main())
```

From asynchronous to parallel programming

By default Python coroutines are executed in the same thread, using **fibers** and **cooperative scheduling** to switch between different execution flows

This paradigm is good as long as a **single event loop** is enough to manage all the events, and it is never busy in **compute-intensive operations** executed in the main program

When compute-intensive tasks are unavoidable, Python allows them to be offloaded to a different **executor** through the `run_in_executor` function

The **ThreadPoolExecutor** offloads the function to a different thread (subject to GIL-induced synchronisations), while the **ProcessPoolExecutor** offloads it to a different process (subject to serialisation/deserialisation overhead)

Data serialisation and deserialisation

When moving from intra-process to inter-process parallelism and distributed computing, the **program context** of the executor differs from the one of the main program

In this setting, **data (de-)serialization** comes into play, and it is one of the **most critical aspects** in practical task-based programming libraries

Almost all task-based libraries rely on a third-party, robust serialization library:

- Python libraries typically rely on **cloudpickle** or **dill**, which augment the native **pickle** module to serialzie and deserialzie practically every Python object
- Rust libraries rely on the **Serde framework**
- Google's **Protocol Buffers** (protobuf) provide APIs for efficient serialization and deserialization in various language, but require users to explicitly provide an object description

A small-scale example: the Monte Carlo method

Monte Carlo methods, or Monte Carlo experiments, are a broad class of computational algorithms that rely on **repeated random sampling** to obtain numerical results.

The underlying concept is to **use randomness to solve problems** that might be deterministic in principle

To demonstrate the usage of task-based programming libraries, we use a simple Python program that **approximates the value of PI** through a Monte Carlo method

```
from random import random

def pi(num_points):
    inside = 0
    for i in range(num_points):
        x, y = random(), random()
        if x**2 + y**2 < 1:
            inside += 1
    return (inside*4 / num_points)

def mean(a, b, c):
    return (a + b + c) / 3

# Estimate three values for PI through a Monte Carlo method
a, b, c = pi(10**6), pi(10**6), pi(10**6)

# Compute the mean of the three estimates
mean_pi = mean(a, b, c)

# Print the results
print("Average: {:.5f}".format(mean_pi))
```

Monte Carlo with `async` / `await`

Adopting the `async` / `await` pattern requires to declare both functions as `async` and to `await` for their completion whenever they are called: it is quite complex to implement

The Monte Carlo simulation is a **compute-intensive** task, so the pure `async` / `await` paradigm is not the best option for this program

```
import asyncio

from random import random

async def pi(num_points):
    inside = 0
    for i in range(num_points):
        x, y = random(), random()
        if x**2 + y**2 < 1:
            inside += 1

    return (inside*4 / num_points)

async def mean(a, b, c):
    return await asyncio.gather(a + b + c) / 3

async def main():
    # Estimate three values for PI through a Monte Carlo method
    a = asyncio.create_task(pi(10**6))
    b = asyncio.create_task(pi(10**6))
    c = asyncio.create_task(pi(10**6))

    # Compute the mean of the three estimates
    mean_pi = asyncio.create_task(mean(a, b, c))

    # Print the results
    print("Average: {:.5f}".format(await mean_pi))

if __name__ == '__main__':
    asyncio.run(main())
```

Monte Carlo with Python executors



UNIVERSITÀ
DI TORINO

Sticking with pure Python APIs, the best option would probably be to rely on **executors** to offload the Pi approximation

This strategy requires a **hybrid approach** (executors for the PI approximation, **async / await** for the mean), which is very complex to be manually programmed (imagine it embedded on a complex data analysis program)

```
import asyncio

from concurrent.futures import ProcessPoolExecutor
from concurrent.futures import ThreadPoolExecutor
from concurrent.futures import wait
from random import random

def pi(num_points):
    inside = 0
    for i in range(num_points):
        x, y = random(), random()
        if x**2 + y**2 < 1:
            inside += 1
    return (inside*4 / num_points)

async def mean(a, b, c):
    return sum(await asyncio.gather(a, b, c)) / 3

async def main():
    loop = asyncio.get_event_loop()
    with ThreadPoolExecutor(max_workers=3) as executor:
        #with ProcessPoolExecutor(max_workers=3) as executor:
        # Estimate three values for PI through a Monte Carlo method
        a = loop.run_in_executor(executor, pi, 10**6)
        b = loop.run_in_executor(executor, pi, 10**6)
        c = loop.run_in_executor(executor, pi, 10**6)

        # Compute the mean of the three estimates
        mean_pi = asyncio.create_task(mean(a, b, c))

        # Print the results
        print("Average: {:.5f}".format(await mean_pi))

if __name__ == '__main__':
    asyncio.run(main())
```

The Dask library

The **Dask library** is a widespread parallel/distributed programming library for Python that extends the **executor** concept, providing a common **Client** interface to a broad range of execution infrastructures (laptop, container orchestrators, batch schedulers, ...)

Dask also provides a **distributed dataframe** abstraction that allows to transparently apply the Pandas DataFrame API to large-scale, distributed datasets

Dask is a **general-purpose parallel library**. However, the DataFrame abstraction makes it suitable for **high-performance data analytics** and **machine learning**



Monte Carlo with Dask

A possible way to optimise the Monte Carlo simulation with Dask is to use the **delayed** abstraction

The Dask delayed function decorates an internal function so that it executes **lazily**. The function and its arguments into a **task graph**

Calling the **compute** function, the task graph is actually computed, returning the final results. The task graph can also be visualised using the **visualize** function

```
import dask
from random import random

def pi(num_points):
    inside = 0
    for i in range(num_points):
        x, y = random(), random()
        if x**2 + y**2 < 1:
            inside += 1
    return (inside*4 / num_points)

def mean(a, b, c):
    return (a + b + c) / 3

# Estimate three values for PI through a Monte Carlo method
a = dask.delayed(pi)(10**6)
b = dask.delayed(pi)(10**6)
c = dask.delayed(pi)(10**6)

# Compute the mean of the three estimates
mean_pi = dask.delayed(mean(a, b, c))

# Print the results
print("Average: {:.5f}".format(mean_pi.compute()))
```

The PyCOMPSs framework

PyCOMPSs, actively developed at BSC, is a framework to execute distributed task-based applications on top of HPC clusters and cloud instances

PyCOMPSs can be used directly as a task-based library, but it also constitutes the **low-level runtime support** of several high-level libraries, including:

- The **Distributed Computing Library (dislib)**, inspired by scikit-learn, which provides common machine learning algorithms that transparently span over distributed data structures
- The **Qdislib**, a Python library for simulating large quantum circuits by splitting them into smaller, independent subcircuits



The Parsl library

Parsl is a native Python task-based programming library developed at the Globus Labs. Parsl wraps Python functions as **apps** using the `@python_app` decorator

Parsl also natively support **external applications** through the `@bash_app` decorator, remote file-based dependencies through a file abstraction, and **dynamic apps**, i.e., apps that generate other apps

Parsl can also be used as the backbone of more advanced computing paradigms, such as:

- **Globus Compute** (formerly **funcX**) for high-performance federated serverless computing
- **Colmena** to seamlessly combine AI and simulation workflows on HPC facilities



Monte Carlo with Parsl



UNIVERSITÀ
DI TORINO

The Parsl API is very straightforward. To transform functions into asynchronous (distributed) tasks, it is sufficient to annotate them with the `@python_app` decorator

Asynchronous functions return a **future object**. To retrieve the actual result, it is necessary to invoke the `result` method

```
import parsl

from parsl.app.app import python_app
from parsl.configs.local_threads import config
from random import random

@python_app
def pi(num_points):
    inside = 0
    for i in range(num_points):
        x, y = random(), random()
        if x**2 + y**2 < 1:
            inside += 1
    return (inside*4 / num_points)

@python_app
def mean(a, b, c):
    return (a + b + c) / 3

# Load Parsl config
parsl.load(config)

# Estimate three values for PI through a Monte Carlo method
a, b, c = pi(10**6), pi(10**6), pi(10**6)

# Compute the mean of the three estimates
mean_pi = mean(a, b, c)

# Print the results
print("Average: {:.5f}".format(mean_pi.result()))
```

The Ray library

Ray is a task-programming library specifically developed for **high-performance AI**. The low-level library, named **Ray Core**, is the runtime support for a suite of AI-oriented libraries:

- **Ray Data** is a scalable data processing library for ML and AI workloads (batch inference, preprocessing, ...)
- **Ray Train** is a scalable machine learning library for distributed training and fine-tuning
- **Ray Tune** is a Python library for experiment execution and hyperparameter tuning at any scale
- **Ray Serve** is a scalable model serving library for building online inference APIs
- **RLlib** is an open source library for production-level, highly scalable, and fault-tolerant reinforcement learning



Monte Carlo with Ray



UNIVERSITÀ
DI TORINO

Also the Ray Core API is quite straightforward. Ray transforms Python functions into asynchronous tasks by wrapping them with the `@ray.remote` decorator

By invoking the `remote` method on the resulting task object (which has the same name of the wrapped function), the task returns a future object

The value of the future can be retrieved by passing it to the `ray.get()` function

```
import ray
from random import random

@ray.remote
def pi(num_points):
    inside = 0
    for i in range(num_points):
        x, y = random(), random()
        if x**2 + y**2 < 1:
            inside += 1
    return (inside*4 / num_points)

@ray.remote
def mean(a, b, c):
    return (a + b + c) / 3

# Initialise Ray
ray.init()

# Estimate three values for PI through a Monte Carlo method
a = pi.remote(10**6)
b = pi.remote(10**6)
c = pi.remote(10**6)

# Compute the mean of the three estimates
mean_pi = mean.remote(a, b, c)

# Print the results
print("Average: {:.5f}".format(ray.get(mean_pi)))
```